Thadomal Shahani Engineering College, Bandra

Department of Artificial Intelligence and Data Science

Name: Krish Makhija	Roll no: 57	Branch: AI&DS
Subject: ASD&D	Batch: T13	Date: 23-07-2026

EXPERIMENT 1: Version Control Operations in Git

This experiment maps to LO2.

LO2: Use Git effectively for version control, source code management, and collaborative team development

Aim:

To learn and perform basic version control operations using Git, including creating a repository, tracking files, committing changes, managing branches, merging branches, and viewing project history, etc.

Theory:

A Version Control System (VCS) is a tool that tracks changes to files over time, allowing software developers to recall specific versions later, revert files back to a previous state, compare changes over time, and collaborate seamlessly on codebases.

VCS is broadly categorized into two types:

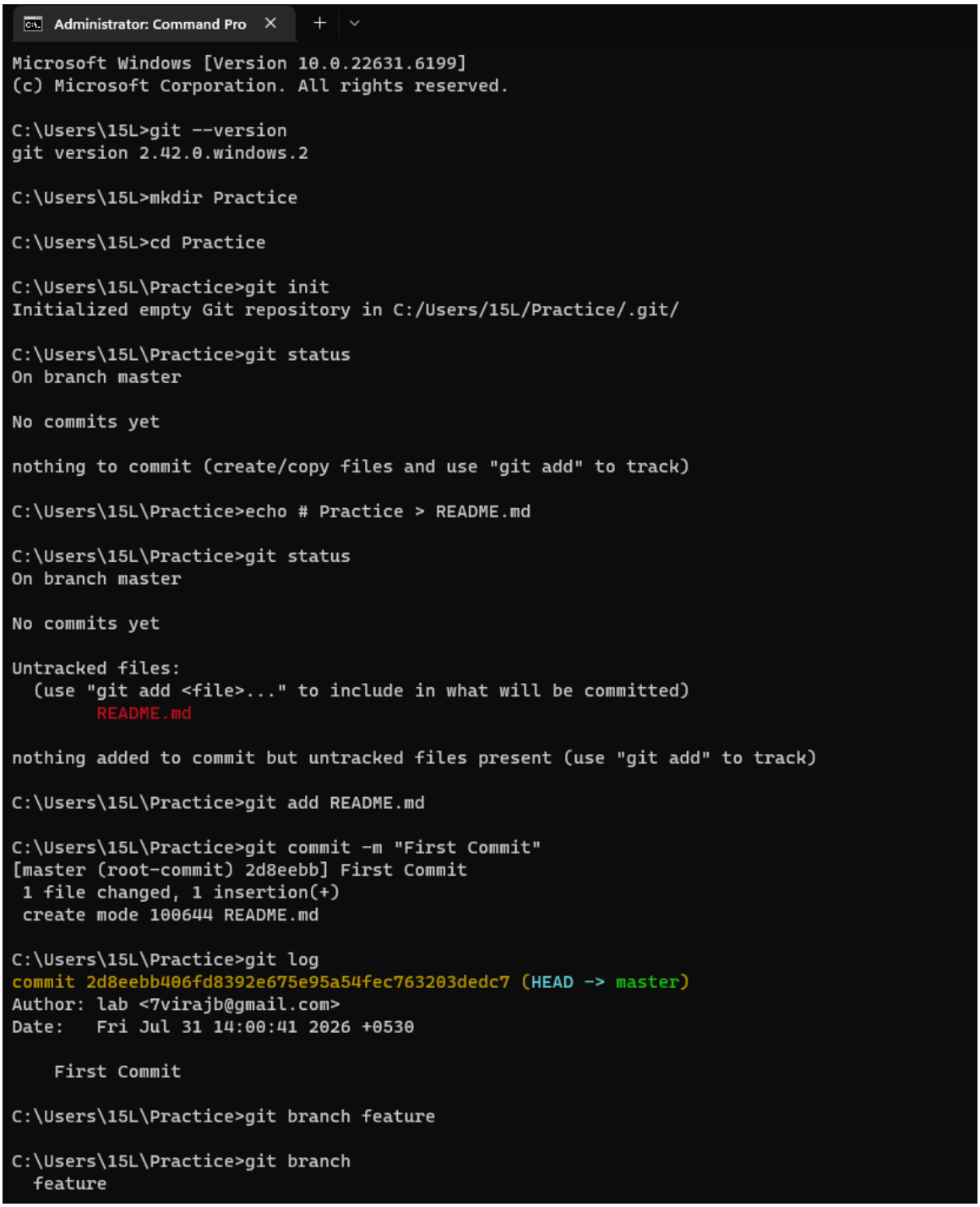
1. Centralized Version Control Systems (CVC**S):** Uses a single central server to store all the versions of files (e.g., SVN, CVS).
2. Distributed Version Control Systems (DVCS): Every developer's local machine holds a full copy of the project repository, including its complete history (e.g., Git, Mercurial).

Git is an open-source, Distributed Version Control System designed to handle projects with speed and efficiency. Unlike older systems, Git stores data as a series of **snapshots** of a mini filesystem. Every time you commit or save the state of your project, Git records what all your files look like at that moment.

A Git project resides in three primary areas on your machine:

1. Working Directory (Working Tree): The local workspace on your disk where you create, edit, or delete files.

2. Staging Area (Index): An intermediate area (a file inside `.git`) that stores information about what changes will go into your next commit.
3. Repository (Git Directory): The hidden `.git` folder where Git stores the metadata and object database for your project.





1. **git –version**

Verifies whether Git is correctly installed on the operating system and prints the installed release version.

2. **mkdir GitPractice**

Creates a new directory named Practice to store the project files.

3. **cd Practice**

Changes the current working directory to the newly created project folder.

4. **git init**

Initializes a new Git repository by creating a hidden .git directory, which stores all version control information.

5. **git status**

Displays the status of the repository, including modified, staged, and untracked files.

6. **echo # Git Practice > README.md**

Creates a new file named README.md that can be used to describe the project.

7. **git status**

Check the repository status again to verify that the newly created file is recognized as an untracked file.

8. **git add README.md**

Stages the specified file, preparing it to be included in the next commit.

9. **git commit -m "First commit"**

Creates the first snapshot of the project and saves the staged changes in the Git repository with a descriptive message.

10. **git log**

Displays the complete history of commits made in the repository.

11. **git branch feature**

Creates a new branch named feature for developing changes independently of the main branch.

12. **git branch**

Lists all branches in the repository and indicates the currently active branch.

13. **git switch feature**

Switches the working directory to the feature branch.

14. **echo "Learning Git" >> README.md**

Appends new content to the README.md file, creating a change that Git can track.

15. **git diff**

Displays the differences between the current working directory and the last committed version of the files.

16. git add .

Stages all modified and newly created files in the current directory.

17. git commit -m "Updated"

Creates another commit that records the latest changes made to the project.

18. git switch master

Switches back to the main branch to continue development or integrate changes.

19. git merge feature

Combines the changes from the feature branch into the main branch.

20. git branch -d feature

Deletes the feature branch after its changes have been successfully merged.

21. git log –oneline

Displays the commit history in a concise, one-line format for quick review.

22. git remote add origin <repository-url>

Adds a remote repository named origin and links the local Git repository to a remote repository hosted on platforms such as GitHub.

23. git remote –v

Displays the configured remote repositories along with their URLs for fetching and pushing changes.

Conclusion:

The experiment successfully demonstrated the basic version of control operations in Git. A local repository was initialized, files were created and committed, branches were managed and merged, and the project's history was reviewed. The repository was then connected to a remote repository, changes were pushed to GitHub, updates were synchronized using pull, and the repository was cloned. These operations demonstrate Git's capabilities for local version control as well as collaboration through remote repositories, making it an essential tool for software development and project management.